Toward the Connection Between Activation Sparsity and Flat Minima

Ze Peng, Jian Zhang, Lei Qi, Yang Gao, and Yinghuan Shi

***Abstract*—The observation that activation sparsity emerges in MLP blocks of standardly trained Transformers offers an opportunity to drastically reduce computation costs without sacrificing performance. To theoretically explain this phenomenon, existing works have shown that activation sparsity does not result from the data properties or data fitting but from the implicit bias of the training process. However, these connections are obtained with strong assumptions (e.g., shallow networks, a small number of training steps, and special training techniques), which cannot be applied to deep models standardly trained with a large number of steps. Different from these works, we find that the flatness of loss landscapes is also closely related to the MLP activation sparsity and can serve as a weaker assumption because it naturally emerges in the standard training of deep networks without the above strong assumptions. Specifically, we find that 1) the MLP activation sparsity equals a ratio between "augmented flatness" (a weighted sum of flatness measures) and the product of the input norm and activation gradient of the MLP. We empirically find that this ratio decreases during training, leading to sparse activations. 2) We also propose the notion of derivative sparsity, which reduces to activation sparsity under ReLU, but further enables pruning in the backward propagation and is more stable than activation sparsity. With the theoretical findings, we can further encourage activation sparsity by decreasing the numerator and increasing the denominator of the ratio: 1) To improve (lower) the flatness, we add different bias vectors to input tokens of MLP blocks to strengthen stochastic gradient noise that drives the model to a flat area. 2) We restrict the lower bound of affine parameters in LayerNorm to increase the input norm of MLPs. 3) To increase the activation sparsity, we propose an activation function JSReLU to encourage the search of parameters with sparse derivatives and sparse activations. These plug-and-play modifications can effectively reduce the ratio and produce sparser activations. Experiments on ImageNet-1K and C4 demonstrate relative improvements of at least 36% on inference sparsity and at least 50% on training sparsity over vanilla Transformers, indicating further potential cost reduction in both inference and training.**

***Index Terms*—Sparsity, flat minima, implicit bias.**

TABLE I
COMPARISON BETWEEN ASSUMPTIONS IN EXISTING EXPLANATIONS OF ACTIVATION SPARSITY AND OURS

	Depth	Training Steps	Training Techniques
[1]	1	1	SGD
[8]	2	All	SAM
[9]	Deep	All	Gaussian Noise
[10]	2, Diagonal	All	SGD
Ours	Deep	All	SGD

I. INTRODUCTION

DEEP neural networks (DNNs) have achieved impressive success on a wide range of tasks. Nevertheless, training and deploying DNNs require substantial computation, time, and especially energy, unlike their biological inspirations, which are highly energy-efficient. One possible reason for this difference is sparsity. In biological neural networks, evidence shows that only a small fraction of neurons spike simultaneously despite the existence of billions of neurons [1]. This activation sparsity is considered a contributing factor to their energy efficiency. However, it was unclear whether artificial DNNs exhibit similar sparsity during inference or training.

A recent work [1] discovers that, in two-layer MLP blocks, activation sparsity emerges *without* explicit regularization, i.e., only a small portion of neurons are activated (with non-zero activation) for a large portion of samples during inference in MLP blocks of feed-forward networks, ResNet, T5 [2], (ReLU) ViT [3], MLP-Mixer [4], and other architectures across various tasks. This finding suggests aggressive neuron pruning during inference without sacrificing performance. For example, if non-activated neurons are ideally skipped, T5's inference FLOPs due to the second linear layers of MLP blocks can be astonishingly reduced by about 90% [1] without performance loss. To *exploit* activation sparsity for faster inference, Mirzadeh et al. [5] observed that the large plateau of ReLU in $(-\infty, 0)$ allows many zero activations, which suggests the renaissance of ReLU in Transformer-based large language models to improve activation sparsity. Several recent works [6], [7] have demonstrated the acceleration by exploiting activation sparsity on large language models. The achievable lossless acceleration of this method is positively correlated with the number of zero activations. Therefore, understanding and improving activation sparsity becomes

Received 18 July 2024; revised 25 March 2026; accepted 25 April 2026. Date of publication 7 May 2026; date of current version 6 August 2026. This work was supported in part by NSFC Project under Grant 62506162, Grant 62506005, and Grant 62192783, in part by Jiangsu Science and Technology Project under Grant BK20251241, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM118, and in part by the "111 Center" under Grant B26023. Recommended for acceptance by R. Giryes. *(Corresponding author: Yinghuan Shi.)*

Ze Peng, Jian Zhang, and Yinghuan Shi are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China, and also with the Institute of Brain-Machine Interface, Nanjing University, Nanjing 210023, China (e-mail: pengze@smail.nju.edu.cn; zhangjian7369@smail.nju.edu.cn; syh@nju.edu.cn).

Lei Qi is with the School of Computer Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: qilei@seu.edu.cn).

Yang Gao is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: gaoy@nju.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2026.3691379, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3691379

critical for further accelerating inference, and possibly training as well.

However, the emergence of sparsity is still not well understood. Several studies [1], [8], [9], [10] have sought to explain it through training dynamics. Li et al. [1] explain activation sparsity in the last layer at the first update step by manually computing gradients. They find that gradients have components that decrease activations in the last MLP block. When sharpness-aware (SA) optimization [11], [12] is used, Andriushchenko et al. [8] manually compute gradients for a shallow 2-layer MLP and find that gradients from the SA objective also contain components that reduce pre-activations. This reduction pushes activations toward zero, inducing sparsity in ReLU networks. [10] considers second-order behaviors of SGD and proves sparsity on diagonal 2-layer MLPs, but the emergence of sparsity in general deep, non-diagonal networks remains conjectural. Bricken et al. [9] find that adding noise to samples improves sparsity. However, this noise is manually imposed and not part of standard augmentations. These studies have improved our understanding of activation sparsity. Specifically, they reveal the roles of training dynamics, especially noise [1], [9], [10] and sharpness reduction [8], in its emergence. Nevertheless, these studies rely on strong assumptions, e.g., *shallow networks* [1], [8], [10], *a small number of training steps* [1], and *additional augmentations [9] or optimization objectives* [8] that are absent from standard training protocols as summarized in Table I. Therefore, a gap remains between these explanations and experiments [1], where all these assumptions are violated, i.e., *activation sparsity emerges in deep networks optimized by standard SGD or variants for millions of steps under standard augmentations*.

To advance understanding, we rebase the emergence of activation sparsity on the inductive bias of flat minima. Flat minima are minima located in wide basins [13] of loss landscape and are favored by SGD because its stochastic gradient noise can drive parameters to escape sharp minima [14], [15], [16]. Flat minima often lead to better generalization of deep neural networks [11], [13], [17], [18], [19]. Flatness is commonly measured by the nuclear [20], [21] or spectral [22], [23], [24] norm of the Hessian of the empirical loss w.r.t. parameters. The lower the measure, the flatter the minima. Flatness has several desirable properties that can help fill these gaps: 1) Flatness allows us to analyze deep (instead of shallow) networks because it is measured w.r.t. all parameters. 2) Under a suitably large learning rate and small batch size, flat minima can be obtained with SGD over many training steps, not only at early updates. 3) Its prevalence under standard training eliminates the need for special training techniques. Therefore, flatness is a more suitable assumption for activation sparsity analysis.

After careful derivation, we find *equalities* linking activation sparsity to a ratio between an augmented form of flatness (i.e., a weighted sum of flatness measures of multiple losses) and the product of MLP input norms and gradient magnitudes on MLP activations in ReLU-activated MLP blocks:

$$\frac{[\text{(Augmented) Flatness}]}{\|[\text{MLP inputs}]\|^2 \times \|[\text{Gradients}]\|^2} = [\text{Activation Sparsity}]. \tag{1}$$

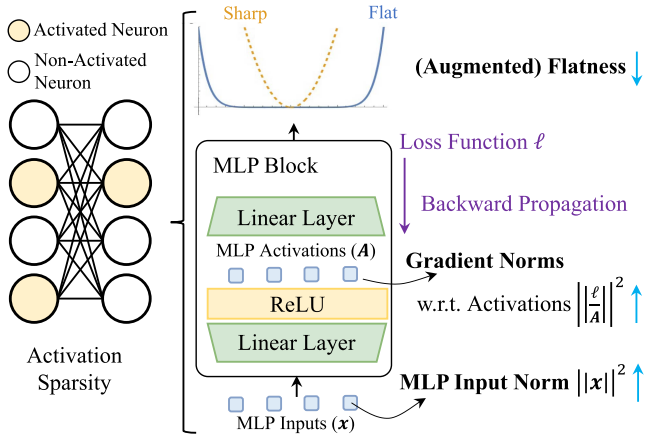


Fig. 1. Overview of theoretical results. We connect activation sparsity with the augmented flatness of losses w.r.t. parameters, gradient norms w.r.t. activations, as well as the squared norms of MLP input tokens.

We empirically measure the augmented flatness and the denominators of the ratio and observe that the denominators increase faster than the augmented flatness. As a result, the ratio decreases and activations become sparse.

We also propose the notion of derivative sparsity, defined as the number of non-zero derivatives of activation functions evaluated at pre-activations (inputs to activation functions) produced during forward propagation. The derivative sparsity equals the activation sparsity when ReLU is used, because the ReLU activation is non-zero if and only if its derivative is non-zero (see Fig. 7). Practically, derivative sparsity enables additional pruning during backward propagation and improves training efficiency. With theoretical and empirical support, we find that derivative sparsity is a more stable notion than activation sparsity under some non-ReLU activation functions. Therefore, derivative sparsity is more closely related to implicit biases in training dynamics and is potentially useful for research on activation sparsity.

The equality between activation sparsity and the ratio indicates that activation sparsity can be improved by decreasing the numerator and increasing the denominator. To this end, we identify factors in vanilla neural networks that hinder reducing this ratio and propose modifications to address them: 1) To decrease the numerator of the ratio, i.e., to improve flatness in MLP blocks, we propose strengthening the gradient noise that MLP blocks are trained with. According to [25], the flatness of MLP blocks can be related to input robustness. Thus, it can be enhanced by noise on MLP input tokens. We rely on gradient noise generated at parameters in shallower layers and forward-propagated to tokens, due to its larger magnitude [14] and alignment with sharp directions [16]. However, for noise on MLP input tokens, we observe that parameter sharing in shallower layers (i.e., shallower layers process all tokens with the same set of parameters) causes noise correlation among different input tokens, resulting in low noise diversity. Besides, we also observe that nonlinearities (e.g., ReLU, attention, *etc*) can suppress noise magnitude when saturated. Therefore, we propose adding different bias vectors to different tokens right before every MLP block so that gradient noise on these bias

vectors are not shared or suppressed by nonlinearities. 2) To increase the MLP input norms in the denominator, we impose lower-bounds on the affine parameters of LayerNorm layers before MLP blocks. 3) With empirical evidence, we find that the training process implicitly decreases the norm of activation derivatives, which benefits derivative and activation sparsity. However, we observe that ReLU with piecewise constant derivatives hinders this implicit optimization: when parameters are perturbed slightly, activation derivatives of ReLU remain constant, creating spurious minima for activation-derivative norms. Therefore, we propose JSReLU with monotonically increasing derivatives to eliminate such spurious minima of activation derivative norms.

In experiments on ImageNet-1K [26] and C4 [2], these plug-and-play modifications bring at least 50% relative improvements in training sparsity and at least 36% in testing sparsity compared to existing naturally emergent sparsity. The contributions of this work are as follows:

- To better understand the emergence of the activation sparsity, we derive equalities between activation sparsity and a ratio between flatness measures and the product of MLP input norms and gradients on MLP activations. We also give empirical results on its decrease;
- We propose the notion of derivative sparsity. We argue for its better stability than activation sparsity by showing that activation sparsity can be lost while derivative sparsity persists under some activation functions;
- We propose three modifications to architectures, activation functions, and normalization layers to improve activation sparsity.
- The proposed modifications achieve at least 36% relative improvements over natural inference sparsity and at least 50% over natural training sparsity in the pretraining of both natural image classification on ImageNet-1K and natural language generalization on C4.

The rest of the manuscript is organized as follows: Section II lists related works; Section III introduces notations; Section IV derives theoretical results and presents empirical evidence for the connection between flatness and sparsity; Section V argues for the value of derivative sparsity; Section VI proposes architectural modifications based on theoretical results and empirical observations; Section VII empirically evaluates the modifications followed by ablation studies; Section VIII summarizes the manuscript, discusses limitations, and outlines potential future work. Proofs can be found in the Appendix. The code for experiments can be found on GitHub.[1]

II. Related Works

In this section, we review related work on the particular roles of neurons in MLP blocks and on understanding the emergence of activation sparsity in these neurons. We also discuss works that further induce and exploit this sparsity for improved computational efficiency. We finally list other notions of emergent sparsity.

[1] https://github.com/cdgyp/sparsity

A. MLP Blocks and Knowledge Neurons

We trace research on activation sparsity in MLP blocks back to the discovery of the relation between MLP blocks and parametric memory. [27] rewrites Transformer MLPs as an unnormalized attention mechanism, where queries are inputs to the MLP block and keys and values come from the first and second weight matrices rather than inputs. In this way, MLP blocks can be viewed as key-value memories. [28] extends this line by detecting how each key-value pair relates to each question in Q&A tasks. By exploiting activation magnitudes and their gradients, they provide a method that surgically manipulates answers to individual questions. These works redirect attention to MLP blocks, which were previously overshadowed by self-attention.

B. Understanding Activation Sparsity

Recently, comprehensive experiments by [1] demonstrate that activation sparsity in MLP blocks is prevalent across architectures and across vision and language tasks. [1] also rules out alternative explanations and attributes activation sparsity solely to training dynamics. The authors explain sparsity theoretically by exploiting properties of random initialization and manually calculating gradients. They find that gradients w.r.t. activations in the last MLP block have positive components that decrease activations and thus push them toward suppression. However, their explanation is restricted to the last layer (due to manual gradient computation) and to the first training step (because their analysis relies on independence between weights and samples, which no longer holds later in training). They also discover that some activation functions, such as $\tanh$, hinder sparsity (see Fig. B.3(c) in [1]), but do not elaborate further. Compared with their explanation, ours applies to all layers and many training steps, and it accounts for the critical role of activation functions in activation sparsity.

Following these empirical discoveries, [8] shows that sharpness-aware (SA) optimization has a stronger bias toward activation sparsity. They explain this theoretically by manually calculating gradients and finding that SA optimization introduces gradient components that reduce activation norms. However, their explanation is still restricted to shallow 2-layer pure MLPs and requires SA optimization, which is not part of standard training practice. Nevertheless, this explanation hints at the role of flatness in the emergence of activation sparsity. Inspired by this, we explain *deep* networks trained by standard SGD or its variants by using flat minima in place of SA optimization.

A more recent work [9] points out that sparsity is resistance to noise. However, the noise is manually imposed and not included in standard data augmentations. We instead consider gradient noise from SGD or other stochastic optimizers. [10] proves sparsity in 2-layer diagonal MLPs and conjectures that a similar process occurs in more general networks. Both works hint at a relation between noise (Gaussian noise added to inputs and stochastic gradient noise) and activation sparsity, and thus to implicit flatness bias.

[29] studies the adversarial robustness of Mixture-of-Experts (MoE) models induced by architecture-imposed sparsity. We are

inspired by this work to relate sparsity to adversarial robustness, although we do so in a reverse and implicit manner.

C. Sparsity Inducing and Exploitation

Although it does not focus on explaining the emergence of activation sparsity in CNNs, [30] boosts activation sparsity through Hoyer regularization [31] and a new activation function, FATReLU, which uses dynamic thresholds between activation and deactivation. They also design algorithms to exploit this sparsity, leading to ≥ 1.75x speedup in CNN inference. Compared with their sparsity-encouraging method, which requires carefully designed threshold-selection procedures, hyperparameters for our theoretically motivated modifications are easier to select. The discontinuity of FATReLU also hinders training from scratch [30], whereas we recommend applying our modifications from scratch to obtain better sparsity and lower *training* cost. [32] encourages activation sparsity in CNNs via explicit L_1 regularization. We instead investigate the emergence of activation sparsity from implicit regularization as demonstrated by [1], and thus rely only on implicit regularization enhanced by our modifications. Nevertheless, our methods are architecturally orthogonal, and we believe combining both could further boost activation sparsity. [5] suggests ReLU for Transformer-based LLMs because smoother activation functions like GELU or SiLU do not provide a large zero-activation plateau where many activations can reside. They also propose a GPU-implemented pruning method, with which inference latency is greatly reduced as sparsity increases.

D. Other Notions of Emergent Sparsity

Another notion of sparsity in hidden features is attention sparsity, i.e., many entries in the attention map are near zero. It is the focus of [33], [34]. For input-side sparsity notions, [35] uses Shapley values to formulate and prove the existence of sparse "symbols," namely small patch groups that are the main contributors to the output of well-trained and masking-robust AI models.

III. Preliminary

Here we introduce the notation. Section III-A presents basic symbols, Section III-B defines the architecture of interest, and Section III-C defines flatness measures and sparsity.

A. Notation and Problem Setting

For $n \in \mathbb{N}$, let $[n] = \{1, 2, \ldots, n\}$. We use bold lowercase letters like $\boldsymbol{x}$ for vectors and bold uppercase letters like $\boldsymbol{X}$ for matrices. When the context is clear, we use x_j for an element of vector $\boldsymbol{x}$, $\boldsymbol{x}_i \in \mathbb{R}^d$ for the *transpose* of a row in matrix $\boldsymbol{X} \in \mathbb{R}^{k\times d}$, and $x_{i,j}$ for an element of matrix $\boldsymbol{X}$. Let $\|\cdot\|_2$ denote the L_2 norm for vectors and $\|\boldsymbol{X}\|_F$ denote the Frobenius norm for matrices. For vectors or matrices, the L_0 pseudo norm $\|\cdot\|_0$ counts their non-zero elements. Let p be the number of parameters. For a parameterized function $f : \mathbb{R}^p \times \mathcal{A} \to \mathcal{B}$, we write f_θ for the partial application of a parameter θ. We also use $f_\theta : \mathcal{A} \to \mathcal{B}$ to indicate $f : \mathbb{R}^p \times \mathcal{A} \to \mathcal{B}$.

We assume classification tasks throughout the analysis. Let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) : i \in [N]\} \subseteq \mathbb{R}^{d_{\text{input}}} \times [C]$ be the *finite* training set. Our focus is solely on activation sparsity within *training* samples, as in [1], [8]. Since the dataset is finite, the analysis involves finite summations rather than integrals. For classification tasks, f_θ outputs a distribution over $[C]$, where $f(c \mid \boldsymbol{x}, \theta)$ denotes the estimated probability of class c for sample $\boldsymbol{x}$ under parameter θ. We define $\ell(f_\theta, (\boldsymbol{x}, y)) := -\log f(y \mid \boldsymbol{x}, \theta)$ as the cross-entropy loss on a sample, and $\hat{\mathcal{L}}(\theta) := \mathbb{E}_{(\boldsymbol{x},y)\sim U\{\mathcal{D}\}}[\ell(f_\theta, (\boldsymbol{x}, y))]$ as the empirical loss on the training set, where $U\{\cdot\}$ denotes the uniform distribution on a non-empty finite set. Our analysis naturally extends to self-supervised objectives in language-model pretraining, since they are essentially token prediction with cross-entropy loss.

B. Architectures

We assume that the network of interest f_θ comprises L MLP blocks interleaved with other structures:

$$f_\theta := \text{softmax} \circ D_\theta \circ \left(\bigcirc_{l=1}^{L} (\text{MLP}_\theta^l \circ G_\theta^l) \right) \circ E, \tag{2}$$

where $\bigcirc$ denotes function composition, E is the input embedding layer, D_θ is the classifier, and $\text{MLP}_\theta^l : \mathbb{R}^{k\times d} \to \mathbb{R}^{k\times d}$ takes a matrix $\boldsymbol{X}^l$ of k d-dimensional tokens and outputs a matrix $\boldsymbol{Z}^l$ of the same shape. G_θ^l encompasses non-MLP structures, including normalization layers, self-attention layers, *etc*. Modules are assumed to use disjoint parameters from θ. Specifically, MLP_θ^l uses parameters from θ to define $(\boldsymbol{K}^l, \boldsymbol{b}^{l,K}) \in \mathbb{R}^{n\times d} \times \mathbb{R}^n$, $(\boldsymbol{V}^l, \boldsymbol{b}^{l,V}) \in \mathbb{R}^{d\times n} \times \mathbb{R}^d$, and computes $\boldsymbol{Z}^l$ by

$$\boldsymbol{P}^l := \boldsymbol{X}^l \times \left(\boldsymbol{K}^l\right)^\top + \boldsymbol{1}_k \times \left(\boldsymbol{b}^{l,K}\right)^\top \in \mathbb{R}^{k\times n}, \tag{3}$$

$$\boldsymbol{A}^l := \sigma\left(\boldsymbol{P}^l\right) \in \mathbb{R}^{k\times n}, \tag{4}$$

$$\boldsymbol{Z}^l := \boldsymbol{A}^l \times \left(\boldsymbol{V}^l\right)^\top + \boldsymbol{1}_k \times \left(\boldsymbol{b}^{l,V}\right)^\top \in \mathbb{R}^{k\times d}, \tag{5}$$

where $\boldsymbol{1}_k \in \mathbb{R}^k$ is the all-1 vector, and $\sigma : \mathbb{R} \to \mathbb{R}$ is the activation function. Here, $\boldsymbol{P}^l$ is the pre-activation matrix and $\boldsymbol{A}^l$ is the activation matrix. $\boldsymbol{K}^l$ and $\boldsymbol{V}^l$ are referred to as key and value matrices, following [27], [28]. Let $\boldsymbol{\theta}_{\text{K}}^l, \boldsymbol{\theta}_{\text{V}}^l$ denote the parameters in $\boldsymbol{K}^l, \boldsymbol{V}^l$, respectively, and let $\boldsymbol{\theta}_{\text{KV}}$ denote the parameters of all MLP matrices. Since the number of tokens affects the analysis, we distinguish networks with different token counts in Definition 1.

Definition 1 (Single- and Multi-Token Scenarios): f is single-token if $k = 1$ and is multi-token if $k > 1$.

According to this definition, feed-forward networks are single-token networks, while CNNs (with receptive fields as tokens), Transformers, and MLP-Mixers are multi-token networks. For clarity, skip connections (e.g., residual and dense connections) are omitted in (2). Nevertheless, our results extend to architectures with skip connections, as detailed in Appendix A.

C. Gradients, Flatness and Sparsities

Given a scalar function g of a matrix or vector $\boldsymbol{X}$, we use $\frac{\partial g}{\partial \boldsymbol{X}} := \left[\frac{\partial g}{\partial x_{i,j}}\right]_{i,j}$ to denote partial derivatives. Our analysis involves non-smooth activation functions such as $\mathrm{ReLU}(x) = \max\{x, 0\}$. With Assumption 1 and Definition 2, we treat undefined derivatives as 0 when computing gradients, supported formally by Lemma A.1 in Appendix A.

Assumption 1: At every $(\boldsymbol{x}, y) \in \mathcal{D}$, the model f and parameter θ satisfy that f_θ is twice continuously differentiable w.r.t. every hidden feature and parameter.

Definition 2: The (zero-assigned) subderivative matrix $\boldsymbol{D}^l$ of MLP^l_θ on sample $(\boldsymbol{x}, y) \in \mathcal{D}$ is $\boldsymbol{D}^l \in \mathbb{R}^{k\times n}$, where

$$d^l_{i,j} := \begin{cases} \sigma'(p^l_{i,j}) & \sigma'(\mathrm{p}^{\mathrm{l}}_{\mathrm{i,j}})\mathrm{exists}, \\ 0 & \text{otherwise}. \end{cases} \tag{6}$$

The Hessian of a scalar function g w.r.t. θ is $\nabla^2_\theta g := \left[\frac{\partial^2 g}{\partial\theta_i \partial\theta_j}\right]_{i,j}$. Specifically, we denote the Hessian of the empirical loss by $H_{\boldsymbol{\theta}_I} := \nabla^2_{\boldsymbol{\theta}_I} \mathbb{E}_{(\boldsymbol{x},y)\sim U\{\mathcal{D}\}}[\ell(f_\theta, (\boldsymbol{x}, y))]$, where $\boldsymbol{\theta}_I$ is any subset of θ. We measure the flatness with $\mathrm{tr}(H_{\boldsymbol{\theta}_I})$ [21], which is the same as $\|H_{\boldsymbol{\theta}_I}\|_*$ used in [20] at minima and lower-bounds the latter in general cases.

Our analysis uses a sum of Hessian traces, as in Definition 3, due to its connection to gradient norms.

Definition 3: The augmented flatness of f_θ on training set $\mathcal{D}$ w.r.t. a subset of parameters $\boldsymbol{\theta}_I$ is

$$\mathrm{AF}_{\boldsymbol{\theta}_I} := \mathrm{tr}\left(H_{\boldsymbol{\theta}_I} + \mathbb{E}_{(\boldsymbol{x},y)\sim U\{\mathcal{D}\}}\left[\frac{\nabla^2_{\boldsymbol{\theta}_I} f(y \mid \boldsymbol{x}, \theta)}{f(y \mid \boldsymbol{x}, \theta)}\right]\right). \tag{7}$$

The augmented flatness combines the trace of the empirical-loss Hessian and the expected Hessian traces of the estimated probabilities of true labels, weighted by those probabilities. The Hessian traces in the second term serve as (weighted) flatness measures for a classification loss that directly uses the estimated probability of the true label as the loss. Therefore, we also consider this term a flatness measure.

Sparsities are defined using hidden features and derivatives.

Definition 4 (Activation Sparsity): The activation sparsity of MLP^l on sample $\boldsymbol{x}$ is $\|\boldsymbol{A}^l\|_0$. The training activation sparsity of MLP^l on training set $\mathcal{D}$ is $\mathbb{E}_{(\boldsymbol{x},y)\sim U\{\mathcal{D}\}}[\|\boldsymbol{A}^l\|_0]$.

Definition 5 (Derivative Sparsity): The derivative sparsity of MLP^l on sample $\boldsymbol{x}$ is $\|\boldsymbol{D}^l\|_0$. The training derivative sparsity of MLP^l on training set $\mathcal{D}$ is $\mathbb{E}_{(\boldsymbol{x},y)\sim U\{\mathcal{D}\}}[\|\boldsymbol{D}^l\|_0]$.

Under ReLU activation, we have $\|\boldsymbol{A}^l\|_0 = \|\boldsymbol{D}^l\|_0$ because for $x \neq 0$, $\mathrm{ReLU}'(x) > 0$ if and only if $\mathrm{ReLU}(x) > 0$.

IV. Connection Between Flatness and Sparsity

In this section, we present a theoretical analysis that reveals the connection between flatness and activation sparsity in MLP blocks. First, we provide the main theorems on the equality between sparsity and the ratio. Next, we support the decrease of the ratio with empirical results.

A. Analysis and Theoretical Results

This subsection presents the main theoretical results connecting flatness and activation sparsity. After noting that gradient norms are linked to augmented flatness and depend on activation derivatives, we first establish the connection between augmented flatness and gradient norms. Then, for single-token scenarios, we decompose the gradient norms to obtain derivative sparsity. Since activation and derivative sparsity are equal for ReLU, we then obtain the connection between augmented flatness and activation sparsity for single-token networks. Finally, we extend the single-token results to multi-token scenarios. Details are in Appendix A.

We first establish an equality between augmented flatness and expected squared gradient norms under cross-entropy loss.

Lemma 1: Under Assumption 1, for any subset of parameters $\boldsymbol{\theta}_I \subseteq \theta$, we have

$$\mathrm{AF}_{\boldsymbol{\theta}_I} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\|\nabla_{\boldsymbol{\theta}_I} \ell(f_\theta, \boldsymbol{s})\|_2^2\right]. \tag{8}$$

To compute gradients, back-propagation uses hidden features and derivatives, thereby linking gradient norms (and augmented flatness) with activation and derivative matrices. This connection allows us to decompose the expected gradient norms of MLP matrices in Lemma 1 with $\boldsymbol{A}^l$ and $\boldsymbol{D}^l$, leading to Lemma 2 (details are in Lemma A.2 of Appendix A).

Lemma 2: Under Assumption 1, for $l \in [L]$, we have

$$\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\left(\boldsymbol{X}^l\right)^\top \left(\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{A}^l} \odot \boldsymbol{D}^l\right)\right\|_F^2\right], \tag{9}$$

$$\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{V}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\left(\boldsymbol{A}^l\right)^\top \frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{Z}^l}\right\|_F^2\right]. \tag{10}$$

To obtain a clearer relationship between flatness and derivative sparsity $\|\boldsymbol{D}^l\|_0$, the number of tokens should be differentiated in the following sections.

B. Single-Token Scenarios

In single-token scenarios, all matrices in Lemma 2 reduce to vectors. This simplification allows us to interchange matrix multiplication and the Frobenius norm, resulting in Lemma 3 (details are in lemma A.3).

Lemma 3: Under Assumption 1, if f_θ is single-token, then for $l \in [L]$,

$$\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{A}^l} \odot \boldsymbol{D}^l\right\|_F^2 \left\|\boldsymbol{X}^l\right\|_F^2\right], \tag{11}$$

$$\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{V}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{Z}^l}\right\|_F^2 \left\|\boldsymbol{A}^l\right\|_F^2\right]. \tag{12}$$

To extract derivative sparsity $\|\boldsymbol{D}^l\|_0$ from Lemma 3, we exploit properties of ReLU and LayerNorm. First, for ReLU networks, when $x \neq 0$, the derivative satisfies $\mathrm{ReLU}'(x) = 1$ if $\mathrm{ReLU}(x) > 0$ and $\mathrm{ReLU}'(x) = 0$ if $\mathrm{ReLU}(x) = 0$. As a result, the subderivatives in $\boldsymbol{D}^l$ act as indicators of whether ReLU neurons are activated. Since indicators are 0-1 valued and the

Frobenius norm of a 0-1 matrix equals its L_0 pseudo norm, one can extract sparsity measured by the L_0 pseudo norm from Frobenius norms with the help of Lemma A.4 in Appendix A. Second, to extract $\|\boldsymbol{X}^l\|_F^2$, we recall that in modern architectures such as Transformers and MLP-Mixer, inputs to MLP blocks are normalized by LayerNorm. If the affine parameters in LayerNorm are turned off and $\epsilon_{\text{LayerNorm}}$ (the value added to the denominator in LayerNorm for numerical stability) is set to 0, then every (and here the only) token in $\boldsymbol{X}$ has L_2 norm $\sqrt{d}$, allowing us to extract $\|\boldsymbol{X}^l\|_F^2$ and leave only $\|\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{A}^l} \odot \boldsymbol{D}^l\|_F^2$ inside the expectation. Theorem 1 summarizes the contributions of LayerNorm and ReLU on activation sparsity. The theorem is proved as an implication of Theorem A.1 in Appendix A.

Theorem 1: Under Assumption 1, if f_θ is used under single-token scenarios, for $l \in [L]$, we have

$$\mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\boldsymbol{D}^l\right\|_0\right] = \frac{\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}}}{\mathbb{E}\left[\|\boldsymbol{X}^l\|_F^2 \left(\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial a^l_{i,j}} \cdot d^l_{i,j}\right)^2 \mid d^l_{i,j} > 0\right]}, \tag{13}$$

$$\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{V}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial \boldsymbol{Z}^l}\right\|_F^2 \left\|\boldsymbol{A}^l\right\|_F^2\right], \tag{14}$$

where the conditional expectation in (13) is taken over $\boldsymbol{s} \sim U\{\mathcal{D}\}, (i,j) \sim U\{[k] \times [n]\}$. If further MLP blocks use ReLU as activation functions, then we have

$$\mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\boldsymbol{A}^l\right\|_0\right] = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\boldsymbol{D}^l\right\|_0\right] \tag{15}$$

$$= \frac{\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}}}{\mathbb{E}\left[\|\boldsymbol{X}^l\|_F^2 \left(\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial a^l_{i,j}}\right)^2 \mid a^l_{i,j} > 0\right]}. \tag{16}$$

If further inputs to MLP blocks are LayerNorm-ed with LayerNorms' affine parameters and $\epsilon_{\text{LayerNorm}}$ turned off, we have

$$\mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\left\|\boldsymbol{A}^l\right\|_0\right] = \frac{\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}}}{d \cdot \mathbb{E}\left[\left(\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial a^l_{i,j}}\right)^2 \mid a^l_{i,j} > 0\right]}. \tag{17}$$

In Theorem 1, derivative sparsity $\|\boldsymbol{D}^l\|_0$ and activation norm $\|\boldsymbol{A}^l\|_F^2$ are connected to augmented flatness w.r.t. key matrices and value matrices, respectively. Activation sparsity $\|\boldsymbol{A}^l\|_0$ is then plugged in via properties of ReLU. As a result, activation sparsity equals a ratio between augmented flatness w.r.t. key matrices and a product related to MLP input norms and magnitudes of gradients w.r.t. activations.

C. Multi-Token Scenarios

When multiple tokens are used, as in CNNs, Transformers, and MLP-Mixers, the derivation is non-trivial because $\boldsymbol{K}^l, \boldsymbol{V}^l$ are shared among tokens and gradients from different tokens may cancel out, preventing results similar to Lemma 3. Fortunately, if we copy the shared $\boldsymbol{K}^l, \boldsymbol{V}^l$ k times (k is the token number) and assign each copy to process exactly one token as in Definition 6, we obtain a single-token network to which we can directly apply the single-token results.

Definition 6: Let f_θ be a neural network defined by (2), then its copied architecture $\tilde{f}$ is defined by

$$\tilde{f}_{\tilde{\boldsymbol{\theta}}} := \mathrm{softmax} \circ D_{\tilde{\boldsymbol{\theta}}} \circ \left(\bigcirc_{l=1}^{L} (\mathrm{kMLP}^l_{\tilde{\boldsymbol{\theta}}} \circ G^l_{\tilde{\boldsymbol{\theta}}})\right) \circ E, \tag{18}$$

where $\mathrm{kMLP}^l_{\tilde{\boldsymbol{\theta}}} : \mathbb{R}^{k\times d} \to \mathbb{R}^{k\times d}$ is defined by

$$\mathrm{kMLP}^l_{\tilde{\boldsymbol{\theta}}}(\tilde{\boldsymbol{X}}^l) = \left[\mathrm{MLP}^{l,i}_{\tilde{\boldsymbol{\theta}}}(\tilde{\boldsymbol{x}}_i^\top)\right]_{i\in[k]}, \tag{19}$$

and $\mathrm{MLP}^{l,i}_{\tilde{\boldsymbol{\theta}}}$ has parameters $(\boldsymbol{K}^{l,i}, \boldsymbol{b}^{l,i,K}) \in \mathbb{R}^{n\times d} \times \mathbb{R}^n$, $(\boldsymbol{V}^{l,i}, \boldsymbol{b}^{l,i,V}) \in \mathbb{R}^{d\times n} \times \mathbb{R}^d$. The copied parameter $\tilde{\boldsymbol{\theta}}$ of θ is constructed by copying parameters of D and G^l, and by copying parameters of MLP^l in f_θ k times to $\mathrm{MLP}^{l,i}$ of $f_{\tilde{\boldsymbol{\theta}}}$. Denote the input and output of $\mathrm{kMLP}^l_{\tilde{\boldsymbol{\theta}}}$ by $\tilde{\boldsymbol{X}}^l$ and $\tilde{\boldsymbol{Z}}^l$, respectively. Let $\tilde{\boldsymbol{\theta}}^l_{\mathrm{K}}$ and $\tilde{\boldsymbol{\theta}}^l_{\mathrm{V}}$ be the parameters in all $\boldsymbol{K}^{l,i}$ and $\boldsymbol{V}^{l,i}$ of layer l, respectively. Let $\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}_I}$ be the augmented flatness in $\tilde{f}_{\tilde{\boldsymbol{\theta}}}$ w.r.t. any subset of parameters $\tilde{\boldsymbol{\theta}}_I$ of $\tilde{\boldsymbol{\theta}}$.

Similar connections between augmented flatness, activation sparsity, and activation norms, as in Theorem 1, can be derived on $\tilde{f}_{\tilde{\boldsymbol{\theta}}}$. Additionally, one can easily verify inductively that $(\tilde{\boldsymbol{X}}^l, \tilde{\boldsymbol{Z}}^l) = (\boldsymbol{X}^l, \boldsymbol{Z}^l)$ on every sample and gradients w.r.t. them are also the same. We then transfer the results on $\tilde{f}_{\tilde{\boldsymbol{\theta}}}$ to f_θ through this equivalence, obtaining Theorem 2, which is an implication of Theorem A.2 in Appendix A.

Theorem 2: Assume that f_θ satisfies Assumption 1 on $\mathcal{D}$. Then we have

$$\mathbb{E}\left[\left\|\boldsymbol{D}^l\right\|_0\right] = \frac{\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_{\mathrm{K}}}}{\mathbb{E}\left[\left\|\boldsymbol{x}_i^l\right\|_2^2 \left(\frac{\partial \ell}{\partial a^l_{i,j}}\right)^2 \left(d^l_{i,j}\right)^2 \mid d^l_{i,j} > 0\right]}, \tag{20}$$

$$\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_{\mathrm{V}}} = \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\sum_{i\in[k]} \left\|\frac{\partial \ell}{\partial \boldsymbol{z}_i^l}\right\|_F^2 \left\|\boldsymbol{a}_i^l\right\|_2^2\right]. \tag{21}$$

where the expectation in (20) is taken over $\boldsymbol{s} \sim U\{\mathcal{D}\}$ and $(i,j) \in U\{[k] \times [n]\}$. If all MLP blocks use ReLU activation functions, we further have

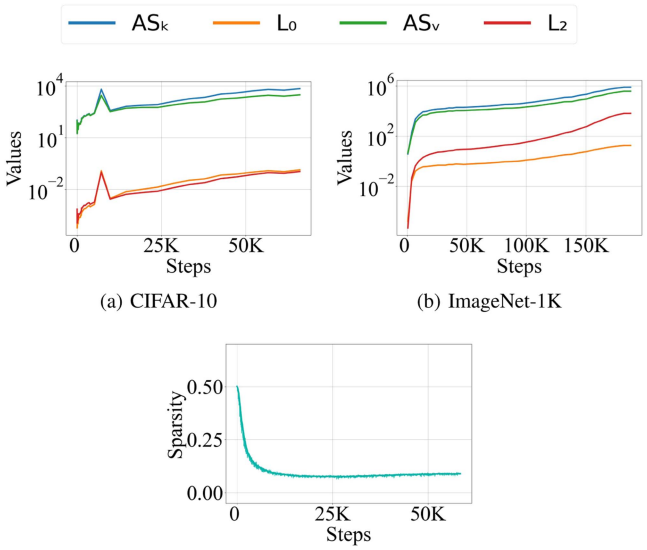
$$\mathbb{E}\left[\left\|\boldsymbol{A}^l\right\|_0\right] = \mathbb{E}\left[\left\|\boldsymbol{D}^l\right\|_0\right] \tag{22}$$

$$= \frac{\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_{\mathrm{K}}}}{\mathbb{E}\left[\left\|\boldsymbol{x}_i^l\right\|_2^2 \left(\frac{\partial \ell}{\partial a^l_{i,j}}\right)^2 \mid a^l_{i,j} > 0\right]}. \tag{23}$$

If further all inputs to MLP blocks are LayerNorm-ed with affine parameters and $\epsilon_{\text{LayerNorm}}$ turned off, we further have

$$\mathbb{E}\left[\left\|\boldsymbol{A}^l\right\|_0\right] = \frac{\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_{\mathrm{K}}}}{d \cdot \mathbb{E}\left[\left(\frac{\partial \ell}{\partial a^l_{i,j}}\right)^2 \mid a^l_{i,j} > 0\right]}. \tag{24}$$

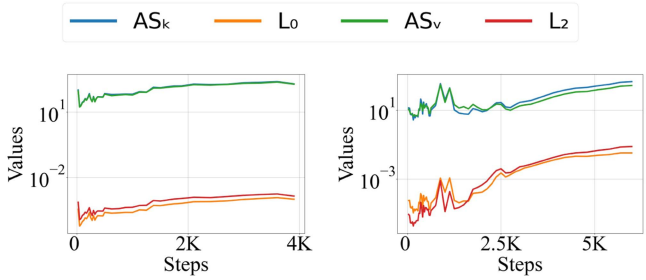
In Theorem 2, we obtain results similar to Theorem 1, except that augmented flatness is defined on the copied version $\tilde{f}_{\tilde{\boldsymbol{\theta}}}$ of f_θ. We will measure this augmented flatness, the denominator



(a) CIFAR-10 (b) ImageNet-1K

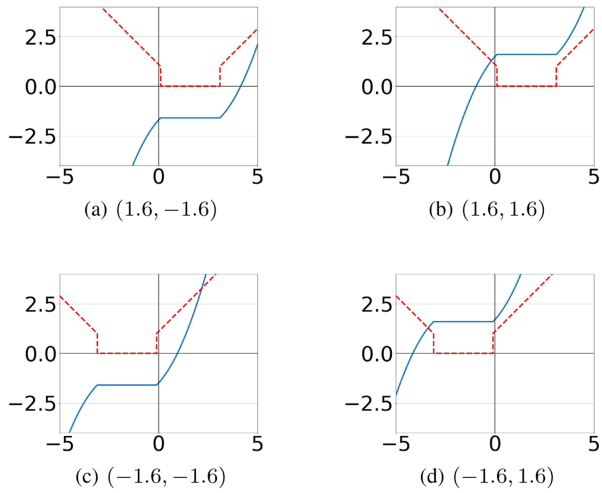
(c) Sparsity on CIFAR-10

Fig. 2. Evolution of $\tilde{\mathrm{AF}}^l_{\tilde{\boldsymbol{\theta}}_\mathrm{K}}, c^l_{L_0}, \tilde{\mathrm{AF}}^l_{\tilde{\boldsymbol{\theta}}_\mathrm{V}}, c^l_{L_2}$ during the entire training on CIFAR-10 and ImageNet-1K as well as the training sparsity on CIFAR-10.



(a) CIFAR-10 (b) ImageNet-1K

Fig. 3. Evolution of $\tilde{\mathrm{AF}}^l_{\tilde{\boldsymbol{\theta}}_\mathrm{K}}, c^l_{L_0}, \tilde{\mathrm{AF}}^l_{\tilde{\boldsymbol{\theta}}_\mathrm{V}}, c^l_{L_2}$ in the first 10 epochs.



(a) $(1.6, -1.6)$ (b) $(1.6, 1.6)$

(c) $(-1.6, -1.6)$ (d) $(-1.6, 1.6)$

Fig. 4. The visualization of weird activation functions with different $(\Delta x, \Delta y)$ and their derivatives, indicated by blue lines and red dashed lines, respectively.

of (23), and the expected average coefficients before activation norms in (21). It turns out that as the training proceeds, the ratio in (23) continues to decrease, leading to activation sparsity. The coefficients before $\|\boldsymbol{a}^l_i\|_2^2$ also increase faster than $\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}_\mathrm{V}}$, so activation norms generally decrease. The augmented flatness in the copied network may seem somewhat distant from the flatness of the original network. To give it a more concrete interpretation, we show that it can be connected to robustness against hidden-feature noise. See Theorem A.2 in Appendix A for details.

D. Empirical Results

We focus on the multi-token case and measure how quantities in Theorem 2 evolve during Transformer training. Specifically, we estimate the augmented flatness $\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_\mathrm{K}}$ and its denominator in (23)

$$c^l_{L_0} := \mathbb{E}\left[\|\boldsymbol{x}^l_i\|_2^2 \left(\frac{\partial \ell}{\partial a^l_{i,j}}\right)^2 \mid a^l_{i,j} > 0\right] \tag{25}$$

as well as the augmented flatness $\tilde{\mathrm{AF}}_{\tilde{\boldsymbol{\theta}}^l_\mathrm{V}}$ and the *expected average* coefficients in (21)

$$c^l_{L_2} := \mathbb{E}_{\boldsymbol{s}\sim U\{\mathcal{D}\}}\left[\frac{1}{k}\sum_{i=1}^{k}\left\|\frac{\partial \ell}{\partial \boldsymbol{z}^l_i}\right\|_2^2\right] \tag{26}$$

for every layer $l \in [L]$ at every checkpoint during training. We train vanilla ViT-Base/16 [3] (with ReLU activation as in [1]) on CIFAR-10 [36] and ImageNet-1K [26] for natural image classification. Experimental details are in Appendix D. The estimates over the full training process are shown in Fig. 2, while those from the initial epochs are shown in Fig. 3. To save space, results are averaged across layers; full layer-wise results are provided in Appendix E. The overall training sparsity on CIFAR-10 is shown in Fig. 2(c), while that on ImageNet-1K is shown in Fig. 8(a).

Throughout training, especially at the beginning, $\tilde{\mathrm{AF}}^l_{\tilde{\boldsymbol{\theta}}_\mathrm{K}}$ increases but more slowly than the denominator. Consequently, the ratio in (23) decreases (as reflected by their difference in the log-scaled plot), leading to the emergence of activation sparsity. Similarly, the activation norms decrease in general, suppressing large activations.

In the later stages of training on ImageNet-1K, activation sparsity deteriorates. We hypothesize that this happens because the denominators $c^l_{L_0}$ increase more slowly. By comparing (23) and (24), we hypothesize that the affine parameters in LayerNorm layers decrease during training, reducing MLP input norms and the denominator $c^l_{L_0}$. To counteract this, we propose lower-bounding the affine parameters in LayerNorm layers, as discussed in Section VI.

V. Discussions on Derivative Sparsity

A byproduct of our analysis is derivative sparsity, which bridges augmented flatness and activation sparsity in Theorems 1 and 2 under ReLU activations. Here, we argue that attention should be paid to the notion of derivative sparsity.

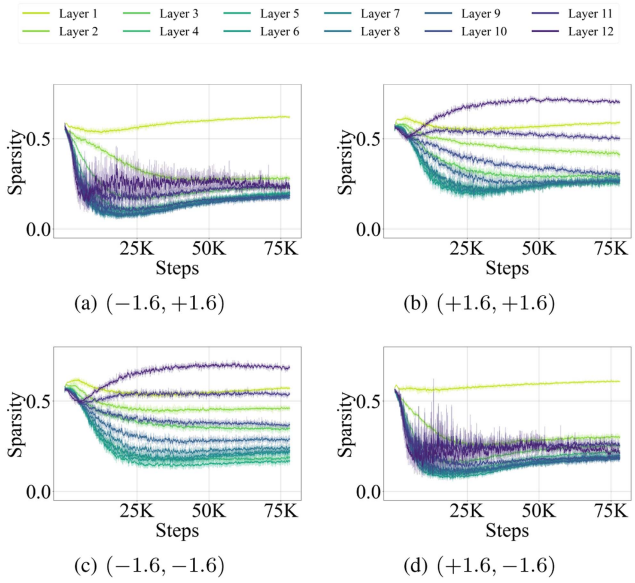


Fig. 5. Derivative sparsity when training ViT-Base/16 with DB-MLP and weird shifted by $\{\pm 1.6\}^2$ on CIFAR-10.

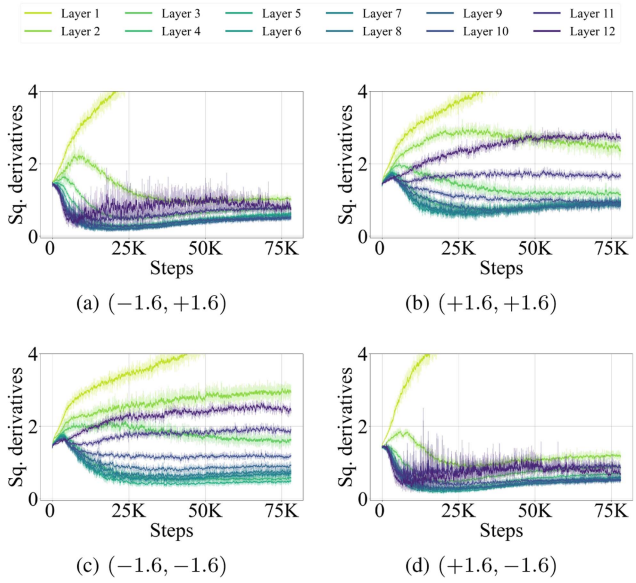


Fig. 6. Squared derivatives during ViT training with weird shifted by different $(\Delta x, \Delta y)$ and DB-MLP on CIFAR-10.

First, derivative sparsity has practical value. In addition to pruning during forward propagation, neurons with zero derivatives backpropagate zero and can be safely pruned during backpropagation. This dual pruning reduces training cost, making derivative sparsity crucial for large-scale training.

Second, the connection between derivative sparsity and flatness is more stable. In Theorem 1 and Theorem 2, the ratio directly and stably equals derivative sparsity. In contrast, the connection to activation sparsity relies on the activation-derivative correlation specific to ReLU, which may not hold for other activation functions. Therefore, derivative sparsity is more closely connected to (augmented) flatness and training dynamics than activation sparsity.

To verify this stability, we empirically demonstrate that derivative sparsity is a more stable notion than activation sparsity under some non-ReLU activation functions. To differentiate activation sparsity and derivative sparsity, we design special activation functions where they are no longer equal:

Definition 7: The base weird function is defined by

$$\mathrm{weird}(x) = \begin{cases} \frac{\mathrm{sgn}(x)}{2}((|x| - w + 1)^2 - 1) & |x| \geq w, \\ 0 & |x| < w. \end{cases} \quad (27)$$

For $\Delta x, \Delta y \in \mathbb{R}$, let $\mathrm{weird}_{\Delta x, \Delta y}(x) = \mathrm{weird}(x - \Delta x) + \Delta y$.

As illustrated in Fig. 4, this family of activation functions has plateaus that are shifted around. On these plateaus, derivatives are zero while activations are nonzero, allowing us to distinguish the two sparsities and compare their stability. Moreover, existing work often interprets activation sparsity through decreasing magnitudes of activations or pre-activations [1], [8]. Motivated by the theoretical stability of derivative sparsity, we ask whether decreases in derivative magnitudes are more stable. To compare these two decreases, we construct $\mathrm{weird}_{1.6,1.6}$ so that stability of the former indicates pre-activation concentration around $x \approx -1.05$, where $\mathrm{weird}_{1.6,1.6}(x) = 0$, while stability of the latter implies concentration on the plateau. One can then compare their stability by where pre-activations concentrate. Similarly, vertical and horizontal shifts allow us to compare stability of decreases in derivative magnitudes against decreases or increases in activations, pre-activations, or their magnitudes. These activation functions have discontinuous derivatives at plateau edges and increasing derivatives beyond the plateaus, designed to amplify these effects in the same spirit as JSReLU defined later in Section VI-C.

We conduct experiments to observe which form of sparsity is preserved. We replace GELU with $\mathrm{weird}_{\Delta x, \Delta y}$, where $(\Delta x, \Delta y) \in \{-1.6, +1.6\}^2$, in ViT-Base/16 and train the model on CIFAR-10 [36]. DB-MLP, defined later in Section VI, is also incorporated to strengthen the effects. The percentage of non-zero derivatives of MLP-block activation functions in training batches is shown in Fig. 5. In most MLP blocks, at least 70% of activation derivatives become zero in deep MLP blocks during training, i.e., derivative sparsity is achieved. At the same time, by construction of the activation function, in most layers at least 70% of activations are non-zero, i.e., activation sparsity is lost. Moreover, as the plateau shifts, derivative sparsity still holds, demonstrating its stability. Similar trends exist for derivative norms in Fig. 6, where squared derivatives decrease in most layers. These results indicate that some previous intermediate characterizations, such as "decreasing activation" [1], [8], are not accurate for general activation functions, because training prefers plateau occupancy and derivative sparsity over smaller activations. Other potential characterizations, such as decreasing or increasing activations, pre-activations, or their norms or absolute values, can also be invalidated by comparing the results of activation functions with differently shifted plateaus. Emphasizing derivative sparsity and derivative norms leads to a better understanding of the mechanisms behind activation sparsity.

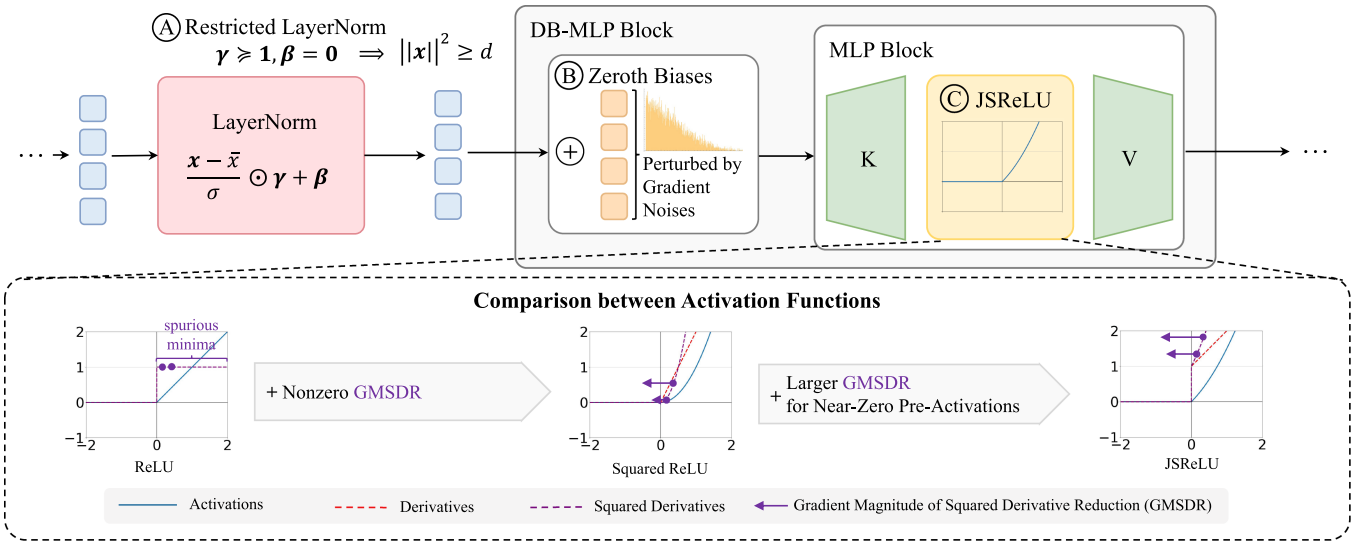


Fig. 7. Overview of methods. In Section VI-A, we restrict affine parameters in LayerNorm layers so that MLP input tokens have large norms. In Section VI-B, we add unshared biases, perturbed by gradient noise, to MLP input tokens, to magnify the gradient noise in MLP input tokens and improve the flatness of MLP blocks. In Section VI-C, we propose a new activation function JSReLU that helps the implicit optimization of reducing the mean squared activation derivatives.

Algorithm 1: Restricted LayerNorm (RLN) for Pretraining.

RLN-PRETRAINING($\mathcal{L}$)
  For $L \in \mathcal{L}$ **do**
    L.weight $\leftarrow$ clamp(L.weight, min $= 1.0$)
  End For

Algorithm 2: Restricted LayerNorm (RLN) for Finetuning.

RLN-FINETUNING($\mathcal{L}, T_{\text{warmup}}, t$)
  For $L \in \mathcal{L}$ **do**
    $\boldsymbol{\gamma} \leftarrow L$.weight
    $\boldsymbol{s} \leftarrow \text{sign}(\text{sign}(\boldsymbol{\gamma}) + 0.1)$
    $p \leftarrow \min(t/T_{\text{warmup}}, 1.0)$
    $\boldsymbol{u} \leftarrow \text{clamp}(\text{abs}(\boldsymbol{\gamma}), \text{min} = p)$
    L.weight $\leftarrow \boldsymbol{s} \odot \boldsymbol{u}$
  End For

VI. Methods

The theoretical and empirical results reveal a connection between activation sparsity and augmented flatness. In addition, properties of normalization layers and activation functions play critical roles in this connection. Therefore, we propose three methods to improve sparsity via flatness, activation functions, and normalization layers, for both pretraining and finetuning.

A. Restricted LayerNorm for Larger Denominator

Theorems 1 and 2 indicate that activation sparsity is related to MLP input norms $\|\boldsymbol{x}_i\|_2^2$ in the ratio denominator. Since MLP inputs are LayerNorm-ed in Transformers, the affine parameters in the LayerNorm layers affect the MLP input norms and the activation sparsity. If these parameters remain active, they can reduce the MLP input norms, thereby decreasing the denominator and increasing the ratio, and harming activation sparsity. To prevent this MLP-input norm reduction, we lower-bound the affine factors so that entries are at least 1. This approach also allows training to adjust and potentially increase them, leading to larger denominators, smaller ratios, and improved sparsity. During pretraining from scratch, we execute Algorithm 1 after every parameter update.

During finetuning, LayerNorm affine factors may not initially be lower-bounded by 1. To address this discrepancy, we freeze LayerNorm biases and gradually increase the lower bound on absolute affine factors during early finetuning until they reach 1, as detailed in Algorithm 2.

B. Zeroth Biases for Larger Stochastic Gradient Noise

It is widely believed that the inductive bias toward flatness stems from gradient noise that makes training escape sharp minima [14], [15]. Therefore, we attempt to improve flatness by increasing the noise. For gradient noise generated at MLP-block parameters themselves, we currently find limited room for further improvement without injecting additional noise. Nevertheless, [25] indicates that MLP-block robustness to inputs is also related to flatness, suggesting potential gains from augmenting noise on MLP input tokens. Given the advantages of gradient noise in magnitude [14] and alignment with sharp local directions [16], we choose to strengthen gradient noise generated at shallower-layer parameters and forward-propagated to MLP input tokens, instead of manually injecting noise. To this end, we identify two insufficiencies in the gradient noise due to the following factors: 1) parameter sharing and 2) nonlinearities. In modern neural networks with extensive parameter sharing, MLP input tokens are produced by the same group of parameters, with minor exceptions such as positional embeddings. As a result, gradient noise on shared parameters introduces correlation between noise on different input tokens. This correlation harms noise diversity on MLP input tokens. Moreover, when forward-propagated through nonlinearities (e.g., ReLU,

Algorithm 3: Restricted Zeroth Biases.

RestrictedZerothBiases($\mathcal{L}, \mathcal{B}, c$)
  For $l = 0 \dots |\mathcal{L}| - 1$ **do**
    $L \leftarrow \mathcal{L}[l]$ // l-th LayerNorm
    $\boldsymbol{B} \leftarrow \mathcal{B}[l]$ // l-th Zeroth Bias right after L
    $\boldsymbol{\gamma} \leftarrow \text{abs}(L.\text{weight})$
    $\boldsymbol{S} \leftarrow c \cdot \begin{bmatrix} \boldsymbol{\gamma} & \boldsymbol{\gamma} & \cdots & \boldsymbol{\gamma} \end{bmatrix}$
    $\boldsymbol{B} \leftarrow \text{clamp}(\boldsymbol{B}, \text{min} = -\boldsymbol{S}, \text{max} = \boldsymbol{S})$
  End For

self-attention), noise is suppressed by saturation, e.g., activation plateaus and softmax saturation in attention. Perturbations to inputs of saturated nonlinearities induce little output response, diminishing noise. Therefore, saturation weakens noise magnitudes on MLP input tokens. Based on these observations, we introduce unshared immediate parameters before MLP blocks to reduce correlation and amplify gradient noise, thereby encouraging training to explore flatter minima w.r.t. $\boldsymbol{\theta}_{\text{KV}}$. Specifically, before feeding an input $\boldsymbol{X}^l$ to an MLP block, we add to it a trainable bias *matrix* $\boldsymbol{B}^l$ of the *same shape* as $\boldsymbol{X}^l$, called Zeroth Bias, forming a Doubly-Biased MLP (DB-MLP):

$$\text{DB-MLP}^l_\theta(\boldsymbol{X}^l) := \text{MLP}^l_\theta(\boldsymbol{X}^l + \boldsymbol{B}^l). \tag{28}$$

Since $\boldsymbol{b}^l_i$ is added only to $\boldsymbol{x}^l_i$ (without sharing) and directly (without nonlinearity), we expect gradient noise on $\boldsymbol{b}^l_i$ to be more diverse and stronger, without correlation and saturation effects.

For both pretraining and finetuning, $\boldsymbol{B}^l$ is initialized to zero to seamlessly adapt pretrained weights. However, since $\boldsymbol{B}^l$ is added after LayerNorm, it may interfere with restricted LayerNorm by canceling large entries of LayerNorm-ed $\boldsymbol{X}^l$ and reducing $\|\boldsymbol{x}^l_i\|_2^2$. To alleviate this conflict, we restrict the magnitude of entries in $\boldsymbol{B}^l$ to allow gradient noise while preventing cancellation. Specifically, if $\boldsymbol{\gamma}^l$ is the affine weight of the LayerNorm right before $\boldsymbol{B}^l$, then we restrict $\boldsymbol{B}^l$ so that $|b^l_{i,j}| \le c \cdot |\gamma^l_j|$ with $c \in (0, 1)$. In this way, we always maintain $\|\boldsymbol{x}^l_i + \boldsymbol{b}^l_i\|_2^2 \ge (1-c)^2 \|\boldsymbol{x}^l_i\|_2^2 \ge (1-c)^2 d$. Since we only lower-bound $|\gamma_j|$ and allow it to exceed 1, this strategy also permits larger gradient-noise magnitudes when $|\gamma_j|$ grows. To implement this strategy, we execute Algorithm 3 after every parameter update.

C. JSReLU *Activation for Better Derivative Sparsity Search*

The coincidence of activation and derivative sparsity under ReLU is beneficial, and we aim to preserve this property. However, ReLU's derivative remains constant after being activated. Its piecewise constant derivative hinders the implicit decrease of derivative norms empirically shown in Fig. 6 of Section V, which benefits derivative sparsity. This is because small parameter perturbations do not change activation derivatives or their norms, creating spurious minima in implicit flatness optimization. To provide local guidance and remove these spurious minima, we make the derivative monotonically increasing so that as activations move toward zero, activation derivatives decrease, immediately benefiting derivative norms. The simplest and most direct modification may be $\text{ReLU}^2(x)$ [37], with monotonically increasing derivative $\max\{2x, 0\}$. However, our tentative experiments show that although large activations become rarer, many small but *non-zero* activations remain, and strictly zero activations become fewer. We hypothesize this is because when the pre-activation $x > 0$ is close to 0, the derivative $2x$ is already very small, providing little drive for further reduction. To address this, we reintroduce the derivative discontinuity of ReLU at 0 using JSReLU, as in Definition 8, so that derivatives remain large near 0 and training is pushed to reduce pre-activations to 0 for augmented flatness. ReLU, ReLU^2, and JSReLU are visually compared in Fig. 7.

Definition 8: $\text{JSReLU} : \mathbb{R} \to \mathbb{R}$ is defined by

$$\text{JSReLU}(x) := \begin{cases} \frac{1}{2}\left((x+1)^2 - 1\right) & x \ge 0 \\ 0 & x < 0. \end{cases} \tag{29}$$

We use JSReLU to replace ReLU from the start of pretraining. To adapt ReLU-pretrained models during finetuning, we linearly mix ReLU with JSReLU and linearly increase the weight of JSReLU during the first few thousand finetuning steps. After this warmup, ReLU is fully removed.

Similar theoretical results can be derived for JSReLU:

$$\mathbb{E}\left[\left\|\boldsymbol{A}^l\right\|_0\right] = \frac{\tilde{\text{AF}}_{\tilde{\boldsymbol{\theta}}^l_{\text{K}}}}{d \cdot \mathbb{E}\left[\left(\frac{\partial \ell}{\partial a_{i,j}} \cdot d_{i,j}\right)^2 \mid a_{i,j} > 0\right]}. \tag{30}$$

The detailed formal results can be found in Appendix C. These results highlight an additional benefit of JSReLU: multiplying gradient magnitudes by $d_{i,j} \ge 1$, which further increases the denominator and decreases the ratio.

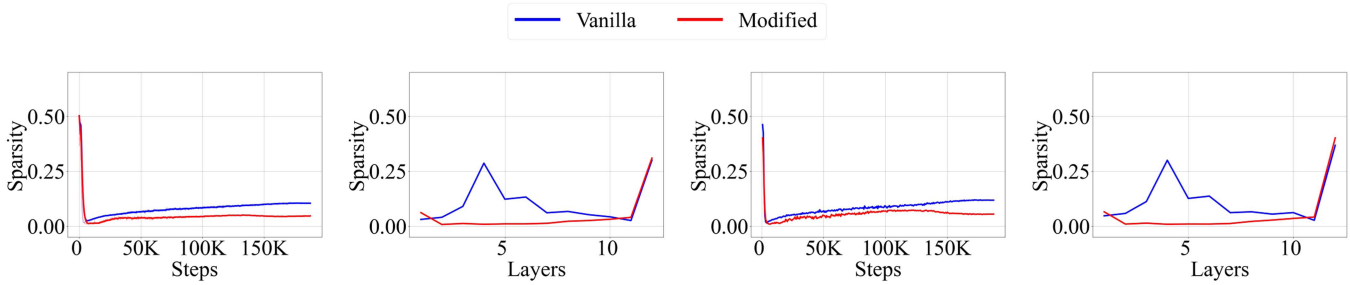
VII. Experiments

A. Evaluation

Methods proposed in Section VI are evaluated on natural image classification and natural language generation tasks. Following [1], we assess and compare sparsity using the percentage of non-zero activations. To better reflect both training and inference costs, we distinguish training sparsity from testing sparsity during pretraining. Since training cost is much lower during finetuning, we focus comparisons primarily on testing sparsity of finetuning. For each layer's MLP block, training sparsity is estimated from training batches every few steps, while testing sparsity is computed over the entire test set at each evaluation. The overall training sparsity is obtained by averaging over steps and layers, and the overall testing sparsity by averaging over layers at the last evaluation. All models are pretrained only once due to their high computational costs. As baselines, we use vanilla (ReLU) vision or language Transformers. Throughout this section, we use warm colors for modified Transformers and cold colors for baselines in plots.

B. Sparsity-Aware Pretraining

For vision tasks, we train ViT [3] or Swin Transformer [38] of various scales on ImageNet-1K [26], Places365 [39], and a LAION-400M [40] subset. Among them, LAION-400M provides language captions for images instead of explicit labels.



(a) Training, stepwise. (b) Training, layerwise. (c) Testing, stepwise. (d) Testing, layerwise.

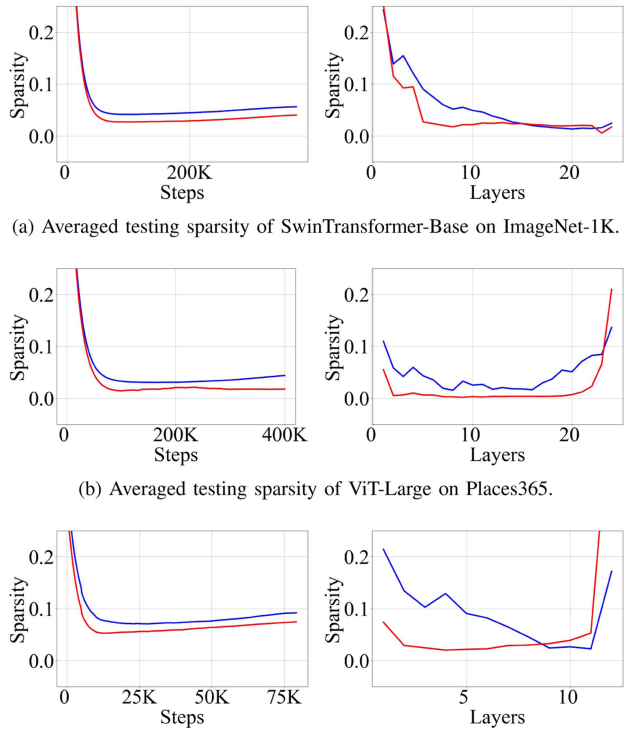
Fig. 8. Training and testing sparsity during training of ViT-Base on ImageNet-1K. (a) and (b) show *training* sparsity while (c) and (d) show *testing* sparsity. (a) and (c) show the evolution of sparsity along the training steps, while (b) and (d) compare sparsity in a layerwise manner. More detailed results can be found in Fig. E.1 of Appendix E.

TABLE II
THE OVERALL TRAINING AND TESTING SPARSITIES OF MODELS PRETRAINED ON VISION TASKS. SYMBOL (↑) INDICATES A HIGHER VALUE IS BETTER, WHILE SYMBOL (↓) MEANS THE OPPOSITE. THE BEST PERFORMANCE AND SPARSITY ARE MARKED **BOLD**.

Setting	Method	Sparsity		Test acc. (↑) or loss (↓)	
		Train (↓)	Test (↓)	Acc@1	Acc@5
ViT-B I.Net	Vanilla	0.104	0.087	**77.35%**	**93.50%**
	Modified	**0.046**	**0.055**	76.77%	93.30%
SwT-B I.Net	Vanilla	0.056	0.057	79.75%	94.73%
	Modified	**0.040**	**0.041**	**79.82%**	**94.90%**
ViT-L P.365	Vanilla	0.049	0.039	**54.81%**	**84.97%**
	Modified	**0.020**	**0.018**	54.60%	84.87%
ViT-B L.40M	Vanilla	0.090	0.092	**0.984**	
	Modified	**0.073**	**0.075**	1.067	

TABLE III
THE OVERALL TRAINING AND TESTING SPARSITIES OF T5-BASE PRETRAINED ON A SUBSET OF C4. SYMBOL (↑) INDICATES A HIGHER VALUE IS BETTER, WHILE SYMBOL (↓) MEANS THE OPPOSITE. THE BEST PERFORMANCE AND SPARSITY ARE MARKED **BOLD**.

Method	Train sparsity (↓)	Test sparsity (↓)	Test loss (↓)
Vanilla	0.302	0.299	4.88
Modified	**0.153**	**0.180**	**4.78**



(a) Averaged testing sparsity of SwinTransformer-Base on ImageNet-1K.

(b) Averaged testing sparsity of ViT-Large on Places365.

(c) Averaged testing sparsity of ViT-Base on a 1/10 subset of LAION-400M.

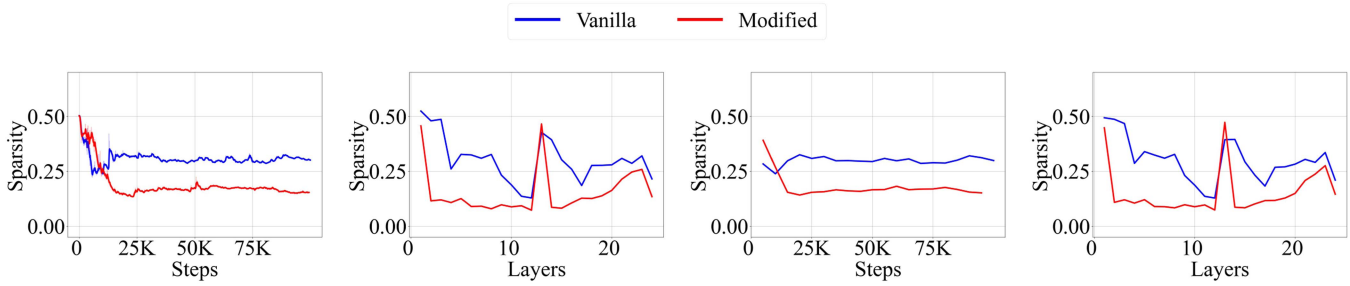
Fig. 9. Testing sparsity of Swin Transformer-Base on ImageNet-1K, ViT-Large on Places365, and ViT-Base on a 1/10 subset of LAION-400M. Testing sparsity is averaged across layers or steps in the same manner as in Fig. 8.

Thus, we train the ViT using CLIP loss [41] with text embeddings from the pretrained CLIP-Base text encoder. We sampled a 1/10 subset of LAION-400M, containing approximately 43 M images. See Appendix D for full details. For language tasks, we train T5-Base [2] on the C4 dataset [2].

To construct the sparsified model, we replace every activation function in vision and language Transformers with JSReLU. Biases in all LayerNorm layers are fixed at 0, while affine factors are lower-bounded by 1 by running Algorithm 1 after every parameter update. Zeroth biases are inserted before every MLP block and restricted by running Algorithm 3 with $c = 0.1$, ensuring that the input to MLP blocks has a large norm $\|\boldsymbol{x}_i^l + \boldsymbol{b}_i^l\|_2^2 \geq 0.81d$. As in [1], T5 is only pretrained for 10^5 steps to save computation.

Training and testing sparsity in vision and language tasks are summarized in Table II and Table III. The evolution of sparsity during training is shown in Figs. 8 to 10. More detailed results can be found in Appendix E. Overall, on the two tasks across all datasets, model architectures, and model scales, our methods achieve $20\% \sim 50\%$ relative improvements on training and testing sparsity compared to the baseline. They achieve the most impressive improvement of approximately 50% on experiments with ViT-B on ImageNet-1K, ViT-Large on Places365, and T5-Base on C4. Despite the drastic improvement in sparsity, the drop in accuracy is minor on vision tasks. In Table III for the NLP task, the testing loss even improves, likely due to the enhanced expressivity from the zeroth biases and the nonlinear JSReLU.

In vision tasks, architecture, model scale, and data distribution strongly impact vanilla and improved sparsity. Deeper networks (ViT-B with 12 MLP blocks vs SwT-B with 24 MLP



(a) Training, stepwise. (b) Training, layerwise. (c) Testing, stepwise. (d) Testing, layerwise.

Fig. 10. Training and testing sparsity during training of T5-Base on C4. (a) and (b) show *training* sparsity while (c) and (d) show *testing* sparsity. (a) and (c) show the evolution of sparsity along the training steps, while (b) and (d) compare sparsity improvements in a layerwise manner. Layers in the encoder are indexed by 1-12 while those in the decoder are indexed by 13-25. More detailed results can be found in Fig. E.5 of Appendix E.

blocks) tend to have stronger vanilla sparsity, possibly due to stronger gradient noise and stronger flatness optimization. The improved sparsity is maintained at the same level, however, leaving the improvement less significant. ViT-L, which is both deeper and wider, leads to equal enhancement on vanilla and improved sparsity, likely because wider networks have more redundancy. Therefore, our methods are the most effective if width and depth are scaled up together, which is often the case in modern practice. Surprisingly, a more diverse and challenging dataset does not severely damage vanilla sparsity, leaving it mainly determined by architecture and model scale. However, improving sparsity becomes difficult, likely because more active neurons are required to fit the data. This contrast is intermediate evidence that vanilla sparsity may contain a lot of redundancy intrinsic to architecture, while our methods can effectively remove it so that fitting difficulty controls sparsity.

From Figs. 8(a), (c) and 9(a) to (c), we observe that modified training improves sparsity through stronger initial sparsification and slower re-densification in the middle and late stages of training. From Figs. 8(b), (d) and 9(a) to (c), one can see that sparsity improvements mainly occur in middle layers, with the exception that the deeper half of SwT-Base layers are naturally sparse and improvements are concentrated in middle layers. Similar observations can be made for T5-Base, except that spatially the sparsity improvements concentrate in the middle layers of both the encoder and the decoder.

Unfortunately, the modified models exhibit sparsity overfitting, while vanilla models' sparsity generalizes well. Nevertheless, the sparsity improvement on the training set is so large that the improvement overwhelms the overfitting.

C. Sparsity-Aware Finetuning

We use the last checkpoints of vanilla Transformers from Section VII-B as pretrained weights and finetune them on ImageNet-1K and C4 for better sparsity, demonstrating a cheaper way to improve sparsity on existing pretrained models.

To enable finetuning, we freeze pretrained parameters and deploy LoRA [42] at weight matrices in MLP blocks and query, key, and value matrices in attention blocks. ViT-Base is tuned for 15 epochs on ImageNet-1K, while T5-Base is tuned for 10,000 steps on the same subset of C4. More details on finetuning can be found in Appendix D.

TABLE IV
TESTING SPARSITY AFTER FINETUNING FOR VIT-BASE ON IMAGENET-1K AND T5-BASE ON C4 FOR BETTER SPARSITY. SYMBOL (↑) INDICATES A HIGHER VALUE IS BETTER, WHILE SYMBOL (↓) MEANS THE OPPOSITE. THE BEST PERFORMANCE AND SPARSITY ARE MARKED **BOLD**. THE SECOND-BEST RESULT IS UNDERLINED.

	ViT-Base		T5-Base	
	Sparsity (↓)	Acc@1 (↑)	Sparsity (↓)	Test Loss (↓)
Before F.T.	**0.087**	**77.35%**	0.299	4.88
Vanilla	0.122	77.17%	0.304	**4.59**
Modified	0.102	74.46%	**0.199**	4.61

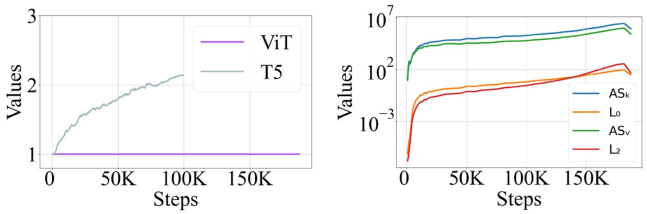
For sparsified models, we add the finetuning version of the proposed modifications with the same configuration as Section VII-B. The warmup of LayerNorm restriction and activation substitution lasts for 3,000 steps. The sparsity before and after finetuning is compared in Table IV.

For ViT, finetuning severely harms both testing sparsity and accuracy. Nevertheless, the modified model still has better sparsity than the baseline after finetuning. However, for T5, the testing sparsity improves by approximately $1/3$ with a minor increase in testing loss.

D. Analysis

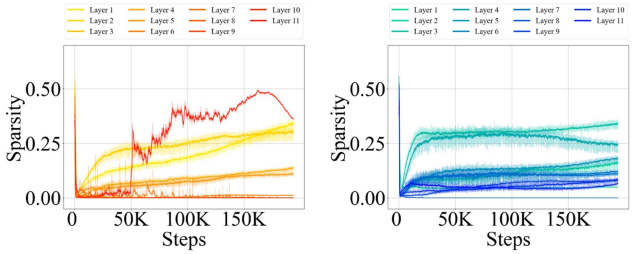
1) Effects of Lower-bounded LayerNorm: The evolution of averaged absolute values of LayerNorm affine factors during sparsity-aware pretraining is shown in Fig. 11(a). It shows that ViT's factors closely adhere to 1, suggesting that without the lower bound they would be much smaller, compromising the denominators $c^l_{L_0}$. For T5, Fig. 11(a) shows little difference with or without the lower bound. The improvement for T5 is due to JSReLU and Zeroth Biases.

To assess how lower-bounding LayerNorm layers mitigate the compromise in denominators $c^l_{L_0}$, we compute the two coefficients of the modified ViT in the same way as Fig. 2, whose results are shown in Fig. 11(b). Note that Fig. 11(b) does not include the denominator increase of JSReLU (see (30)) and only displays the increase due to lower-bounded LayerNorms in the sparsified ViTs. It can be seen that $c^l_{L_0}$ becomes larger compared



(a) (b)

Fig. 11. The role of lower-bounding LayerNorm layers. (a) displays the averaged absolute values of affine parameters in LayerNorm layers, during the pretraining of the modified models. (b) displays how the $c^l_{L_0}, c^l_{L_2}$ evolve during the pretraining of the modified ViT. Full layerwise results can be found in Appendix E.



(a) Modified, overall: 0.115. (b) Vanilla, overall: 0.130.

Fig. 12. The training sparsity of ViT trained with only (unrestricted) DB-MLP and JSReLU, without restrictions on LayerNorm layers and DB-MLP. "Overall" indicates their overall training sparsity across layers and steps.

to Fig. 2(b). As a result, the ratio in Theorem 2 becomes smaller, and better activation sparsity is gained, given that the augmented flatness remains similar.

2) Ablation on Lower-bounded LayerNorm: To assess the contribution of the lower bound among the proposed methods for ViT, we present results from tentative experiments with Zeroth Biases and JSReLU on ImageNet-1K, where LayerNorm affine factors and Zeroth Biases are unbounded. Besides removing restrictions, we also used a larger gradient clipping norm of up to 10, hoping for better flatness. However, this leads to unstable training, likely due to the larger derivatives of JSReLU. Other setting differences from Section VII-B are listed in Appendix D. The results are shown in Fig. 12.

The results show that while the training sparsity improves, the degree of improvement is much smaller. The preference for augmented flatness on L_2 terms also appears much earlier, leading to inferior training sparsity measured by the L_0 pseudo norm in later training. This indicates that lower-bounded LayerNorm is critical for improving sparsity in ViT, as are DB-MLP and JSReLU.

VIII. Conclusion, Limitation, and Future

In this paper, we focus on the emergence and enhancement of activation sparsity in MLP blocks. Regarding emergence, we derive equalities linking activation sparsity to a ratio between an augmented form of flatness and a denominator related to MLP input norms and gradient magnitudes of MLP activations. By empirically measuring the augmented flatness, the denominator, and the ratio, we observe that the denominator increases faster than the augmented flatness, causing the ratio to decrease as training proceeds and leading to activation sparsity. We propose derivative sparsity, which allows pruning during backward propagation and is more stable than activation sparsity. Based on the theoretical roles of flatness, activation functions, and normalization layers, we propose three simple plug-and-play methods to improve sparsity. Experimental results demonstrate that sparsity-aware pretraining substantially improves both training and testing sparsity compared with baseline Transformers on natural image classification and natural language generation tasks. This improvement offers strong potential cost reductions during both pretraining and inference.

Activation sparsity is connected to augmented flatness, which may attract more research attention towards understanding the emergence of flatness. However, our conclusion on sparsity emergence is supplemented by empirical results. Theoretical explanations for why the measured quantities behave as observed in our findings remain unknown. Lastly, the enhanced sparsity exhibits overfitting to the training set. Developing methods to alleviate this overfitting could further improve inference efficiency.

We discuss conjectures and potential future studies on the above limitations. For a more complete picture on sparsity emergence, we may need to rewrite (17) into $\mathrm{AF}_{\boldsymbol{\theta}^l_{\mathrm{K}}} = \mathbb{E}[\|\boldsymbol{A}^l\|_0] \cdot d \cdot \mathbb{E}[(\frac{\partial \ell(f_\theta, \boldsymbol{s})}{\partial a^l_{i,j}})^2 \mid a^l_{i,j} > 0]$. This indicates that better sparsity can lead to better flatness, and the training *may* implicitly optimize sparsity to gain flatness. However, as observed in the above equality, flatness is not solely determined by sparsity, and training may optimize flatness by decreasing the other factor and totally ignoring sparsity. We argue that sparsity optimization is not ignorable, if not dominant, in the flatness optimization. At least in pure MLPs, the other factor is only determined by the sparsity of other layers and weights (especially the spectral properties of the weights). Studies on training dynamics show that the ubiquitous small initialization and (S)GD encourage the weights to be spectrally auto-aligned [43], [44], [45], [46], which are exactly the flattest solution [47]. Therefore, the other factor has been fully optimized by (S)GD, and the only factor that can be further optimized is sparsity. This explains why sparsity must be enhanced when SGD seeks flatness. Additionally, flatness is mainly optimized through escape from sharp minima driven by stochastic gradient noise [16], [48]. It is unlikely that stochastic escaping solely favors the other factor. Minima with sparsity will be visited and can trap training due to better flatness. Nevertheless, this is only a sketchy conjecture, and we leave the rigorous proof for future studies. For sparsity overfitting, if we treat sparsity as a special loss, information-theoretic or PAC-Bayesian generalization theories that impose few assumptions on loss can help. Since sparsity percentage $\|\boldsymbol{D}^l\|_0/kn \in [0, 1]$ is bounded and thus satisfies the sub-Gaussian condition in these bounds [49], [50], the gap between training and testing sparsity percentages can be bounded by $\sqrt{I(\mathcal{D};\theta)/2|\mathcal{D}|}$, where $I(\mathcal{D};\theta)$ is the mutual information between the randomly drawn training dataset and the learned parameters. See Theorem F.1 in Appendix F for formal results. More advanced bounds [51], [52],

[53], [54], [55] can also be applied. Theoretically, this places sparsity overfitting in a unified framework with mainstream studies of loss or accuracy overfitting and allows progress to transfer across both fields. For algorithms, regularization techniques based on information-theoretic or PAC-Bayesian theories may help mitigate sparsity overfitting, which we leave for future studies.

REFERENCES

[1] Z. Li et al., "The lazy neuron phenomenon: On emergence of activation sparsity in transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–35.

[2] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[3] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–21.

[4] I. O. Tolstikhin et al., "MLP-Mixer: An all-MLP architecture for vision," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 24261–24272.

[5] S. I. Mirzadeh et al., "ReLU strikes back: Exploiting activation sparsity in large language models," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 1–23.

[6] Z. Liu et al., "Deja vu: Contextual sparsity for efficient LLMs at inference time," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 22137–22176.

[7] J. Liu, P. Ponnusamy, T. Cai, H. Guo, Y. Kim, and B. Athiwaratkun, "Training-free activation sparsity in large language models," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 1–19.

[8] M. Andriushchenko, D. Bahri, H. Mobahi, and N. Flammarion, "Sharpness-aware minimization leads to low-rank features," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 47032–47051.

[9] T. Bricken, R. Schaeffer, B. Olshausen, and G. Kreiman, "Emergence of sparse representations from noise," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 3148–3191.

[10] M. Andriushchenko, A. V. Varre, L. Pillaud-Vivien, and N. Flammarion, "SGD with large step sizes learns sparse features," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 903–925.

[11] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–20.

[12] K. Wen, T. Ma, and Z. Li, "How sharpness-aware minimization minimizes sharpness?," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–94.

[13] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–16.

[14] U. Simsekli, L. Sagun, and M. Gurbuzbalaban, "A tail-index analysis of stochastic gradient noise in deep neural networks," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 5827–5837.

[15] P. Zhou, J. Feng, C. Ma, C. Xiong, S. C. H. Hoi, and W. E., "Towards theoretically understanding why SGD generalizes better than Adam in deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, Red Hook, NY, United States: Curran Associates, Inc., 2020, vol. 33, pp. 21285–21296.

[16] L. Wu, M. Wang, and W. Su, "The alignment property of SGD noise and how it helps select flat minima: A stability analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 4680–4693.

[17] P. Izmailov, A. Wilson, D. Podoprikhin, D. Vetrov, and T. Garipov, "Averaging weights leads to wider optima and better generalization," in *Proc. 34th Conf. Uncertainty Artif. Intell.*, 2018, pp. 876–885.

[18] Y. Jiang, B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio, "Fantastic generalization measures and where to find them," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–33.

[19] C. Tan, J. Zhang, J. Liu, and Y. Gong, "Sharpness-aware lookahead for accelerating convergence and improving generalization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10375–10388, Dec. 2024.

[20] A. Achille and S. Soatto, "Emergence of invariance and disentanglement in deep representations," *J. Mach. Learn. Res.*, vol. 19, no. 50, pp. 1–34, 2018.

[21] A. Orvieto, H. Kersting, F. Proske, F. Bach, and A. Lucchi, "Anticorrelated noise injection for improved generalization," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 17094–17116.

[22] J. Cha et al., "Swad: Domain generalization by seeking flat minima," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 22405–22418.

[23] X. Zhang, R. Xu, H. Yu, H. Zou, and P. Cui, "Gradient norm aware minimization seeks first-order flatness and improves generalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 20247–20257.

[24] X. Zhang, R. Xu, H. Yu, Y. Dong, P. Tian, and P. Cui, "Flatness-aware minimization for domain generalization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 5189–5202.

[25] H. Petzka, M. Kamp, L. Adilova, C. Sminchisescu, and M. Boley, "Relative flatness and generalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 18420–18432.

[26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[27] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Stroudsburg, PA, USA: Association for Computational Linguistics, 2021, pp. 5484–5495.

[28] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei, "Knowledge neurons in pretrained transformers," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguist.*, Stroudsburg, PA, USA: Association for Computational Linguistics, 2022, pp. 8493–8502.

[29] J. Puigcerver, R. Jenatton, C. Riquelme, P. Awasthi, and S. Bhojanapalli, "On the Adversarial Robustness of Mixture of Experts," in *Proc. Adv. Neural Inf. Process. Syst.*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., Red Hook, NY, United States: Curran Associates, Inc., 2022, vol. 35, pp. 9660–9671.

[30] M. Kurtz et al., "Inducing and exploiting activation sparsity for fast inference on deep neural networks," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 11, 2020, pp. 5533–5543.

[31] P. O. Hoyer, "Non-negative matrix factorization with sparseness constraints," *J. Mach. Learn. Res.*, vol. 5, no. 9, 2004, pp. 1457–1469.

[32] G. Georgiadis, "Accelerating convolutional neural networks via activation map compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 7085–7095.

[33] G. M. Correia, V. Niculae, and A. F. T. Martins, "Adaptively sparse transformers," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process.*, Stroudsburg, PA, USA: Association for Computational Linguistics, 2019, vol. 11, pp. 2174–2184.

[34] K. Lin et al., "SwinBERT: End-to-end transformers with sparse attention for video captioning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 17949–17958.

[35] Q. Ren, J. Gao, W. Shen, and Q. Zhang, "Where we have arrived in proving the emergence of sparse interaction primitives in DNNs," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–13.

[36] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Tech. Rep., 2009, pp. 1–60.

[37] D. So, W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le, "Searching for efficient transformers for language modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, Red Hook, NY, United States: Curran Associates, Inc., 2021, vol. 34, pp. 6010–6022.

[38] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 10012–10022.

[39] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 6, pp. 1452–1464, Jun. 2018.

[40] C. Schuhmann et al., "Laion-400M: Open dataset of clip-filtered 400 million image-text pairs," 2021, *arXiv:2111.02114*.

[41] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.

[42] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–26.

[43] Z. Ji and M. Telgarsky, "Gradient descent aligns the layers of deep linear networks," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 1–9.

[44] D. Stöger and M. Soltanolkotabi, "Small random initialization is akin to spectral learning: Optimization and generalization guarantees for overparameterized low-rank matrix reconstruction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 23831–23843.

[45] M. Soltanolkotabi, D. Stöger, and C. Xie, "Implicit balancing and regularization: Generalization and convergence guarantees for overparameterized asymmetric matrix sensing," in *Proc. 36th Annu. Conf. Learn. Theory*, 2023, pp. 5140–5142.

[46] C. C. J. Dominé et al., "From lazy to rich: Exact learning dynamics in deep linear networks," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 1–38.

[47] R. Mulayoff and T. Michaeli, "Unique properties of flat minima in deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 7108–7118.
[48] T. Mori, L. Ziyin, K. Liu, and M. Ueda, "Power-law escape rate of SGD," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 15959–15975.
[49] A. Xu and M. Raginsky, "Information-theoretic analysis of generalization capability of learning algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1–10.
[50] P. Alquier, "User-friendly introduction to PAC-Bayes bounds," 2023, *arXiv: 2110.11216*.
[51] T. Steinke and L. Zakynthinou, "Reasoning about generalization via conditional mutual information," in *Proc. 33th Annu. Conf. Learn. Theory*, 2020, pp. 3437–3452.
[52] A. Asadi, E. Abbe, and S. Verdu, "Chaining mutual information and tightening generalization bounds," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31, pp. 1–10.
[53] H. Hafez-Kolahi, Z. Golgooni, S. Kasaei, and M. Soleymani, "Conditioning and processing: Techniques to improve information-theoretic generalization bounds," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 16457–16467.
[54] Z. Wang and Y. Mao, "Tighter information-theoretic generalization bounds from supersamples," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 36111–36137.
[55] Z. Peng, J. Zhang, Y. Wang, L. Qi, Y. Shi, and Y. Gao, "Leveraging flatness to improve information-theoretic generalization bounds for SGD," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 1–15.

Ze Peng is currently working toward the PhD degree with the Department of Computer Science, Nanjing University, Nanjing, China. His research interests include deep learning theory, particularly non-convex optimization and generalization.

Jian Zhang is currently a postdoctoral researcher with the School of Intelligence Science and Technology, Nanjing University, Suzhou, China. His research interests include machine learning, computer vision and domain generalization and their applications in medical imaging and materials science.

Lei Qi is currently an associate professor with the School of Computer Science and Engineering, Southeast University, China. His research interests include some ML methods, such as domain adaptation, semi-supervised learning, unsupervised learning, and meta-learning. His research focuses on person re-identification and image segmentation.

Yang Gao is currently a professor with the Department of Computer Science and Technology, Nanjing University. He is directing the Reasoning and Learning Research Group with Nanjing University. He has authored or coauthored more than 100 papers in top-tier conferences and journals. His research interests include artificial intelligence and machine learning. He serves as program chair and area chair for many international conferences.

Yinghuan Shi received the BSc and PhD degrees in computer science, Nanjing University, in 2007 and 2013, respectively. He is currently a professor with the Department of Computer Science and Technology, Nanjing University. He is also affiliated with National Institute of Healthcare Data Science, Nanjing University, and Institute of Brain-Machine Interface. He has authored or coauthored more than 80 papers in CCF-A Conference and *IEEE Transactions*. His research interests include machine learning, pattern recognition, and medical image analysis.